# Null Object

Null Object · Safe logger when monitoring disabled · Build with Waqar

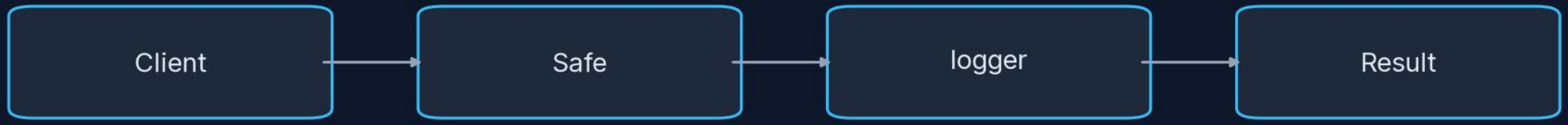
# Today's Hook

- Problem: Safe logger when monitoring disabled
- Pattern: Null Object — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

# The Problem

- Safe logger when monitoring disabled
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

# Structure




Null Object: Safe logger when monitoring disabled

# Real-World Scenario

- Running example: Safe logger when monitoring disabled
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

# Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.nullobject;

import com.buildwithwaqar.patterns.enterprise.nullobject.*;
/** Problem: disable logging without null checks at every call site. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Null Object problem: optional logger ===");
    System.out.println(new OrderService(null).place("o1"));
    var sink = new StringBuilder();
    System.out.println(new OrderService(new ConsoleLogger(sink)).place("o2"));
    System.out.println("log=" + sink.toString().trim());
  }
}
```

# Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

# Solution

```java
package com.buildwithwaqar.patterns.enterprise.nullobject;

interface Logger {
    void info(String msg);
}

final class ConsoleLogger implements Logger {
    private final StringBuilder sink;
    ConsoleLogger(StringBuilder sink) { this.sink = sink; }
    public void info(String msg) { sink.append(msg).append('\n'); }
}

final class NullLogger implements Logger {
    static final NullLogger INSTANCE = new NullLogger();
    public void info(String msg) { /* no-op */ }
}

/** Prefer Null Object over null checks at call sites. */
public final class OrderService {
    private final Logger logger;

    public OrderService(Logger logger) {
        this.logger = logger == null ? NullLogger.INSTANCE : logger;
    }
```

# Solution (2)

```
    public String place(String id) {
        logger.info("placing " + id);
        return "ok:" + id;
    }
}
```

# Runnable main

```
package com.buildwithwaqar.patterns.enterprise.nullobject;

import com.buildwithwaqar.patterns.enterprise.nullobject.*;
/** Problem: disable logging without null checks at every call site. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Null Object problem: optional logger ===");
    System.out.println(new OrderService(null).place("o1"));
    var sink = new StringBuilder();
    System.out.println(new OrderService(new ConsoleLogger(sink)).place("o2"));
    System.out.println("log=" + sink.toString().trim());
  }
}
```

# Walkthrough Output

```
=== Null Object problem: optional logger ===
ok:o1
ok:o2
log=placing o2
```

# Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

# Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

# Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Null Object.

# Related Patterns

- Contrast Null Object with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

# Recap

- Problem → naive → Null Object → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.